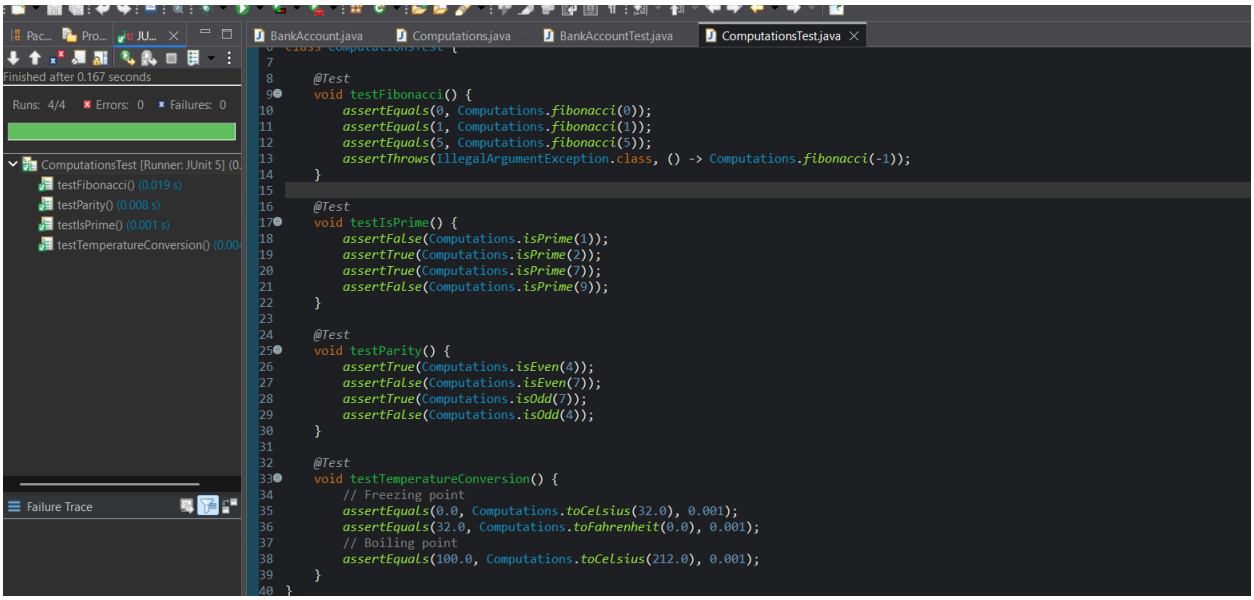

Finished after 0.185 seconds
Runs: 8/8
Errors: 0
Failures: 0
BankAccountTest [Runner: JUnit 5] (0.0
testNegativeInitialBalance() (0.021
testStringEquality() (0.002 s)
testWithdraw() (0.001 s)
testOverdraft() (0.001 s)
testInvalidDeposit() (0.001 s)
testArrayValues() (0.005 s)
testDeposit() (0.001 s)
testTransfer() (0.000 s)
Failure Trace
92 account.transfer(otherAccount, 30.0);
94 assertEquals(70.0, account.getBalance(), "Source account should have 70 left");
95 // Verify the destination account was credited
96 assertEquals(80.0, otherAccount.getBalance(), "Destination account should now have 80");
97 }
99 /**
100 * Fail if any value in an array is less than 20.
101 */
102 @Test
103 void testArrayValues() {
104 int[] values = {25, 30, 21, 40};
105 for (int val : values) {
106 // assertTrue will cause the test to fail if it encounters a value like 19
107 assertTrue(val >= 20, "Value " + val + " was less than 20");
108 }
109 }
111 /**
112 * Pass only if two strings contain the same characters.
113 */
114 @Test
115 void testStringEquality() {
116 String strOne = "hello";
117 String strTwo = new String("hello");
119 // assertEquals checks content equality (like strOne.equals(strTwo))
120 assertEquals(strOne, strTwo, "Strings must contain identical characters to pass");
121 }
123 // Question 4: Yes, the other methods will still execute if the first test fails.
124 }


Finished after 0.167 seconds
Runs: 4/4
Errors: 0
Failures: 0
ComputationsTest [Runner: JUnit 5] (0.
testFibonacci() (0.019 s)
testParity() (0.008 s)
testIsPrime() (0.001 s)
testTemperatureConversion() (0.00
Failure Trace
BankAccount.java
Computations.java
BankAccountTest.java
ComputationsTest.java
8 @Test
9 void testFibonacci() {
10 assertEquals(0, Computations.fibonacci(0));
11 assertEquals(1, Computations.fibonacci(1));
12 assertEquals(5, Computations.fibonacci(5));
13 assertThrows(IllegalArgumentException.class, () -> Computations.fibonacci(-1));
14 }
16 @Test
17 void testIsPrime() {
18 assertFalse(Computations.isPrime(1));
19 assertTrue(Computations.isPrime(2));
20 assertTrue(Computations.isPrime(7));
21 assertFalse(Computations.isPrime(9));
22 }
24 @Test
25 void testParity() {
26 assertTrue(Computations.isEven(4));
27 assertFalse(Computations.isEven(7));
28 assertTrue(Computations.isOdd(7));
29 assertFalse(Computations.isOdd(4));
30 }
32 @Test
33 void testTemperatureConversion() {
34 // Freezing point
35 assertEquals(0.0, Computations.toCelsius(32.0), 0.001);
36 assertEquals(32.0, Computations.toFahrenheit(0.0), 0.001);
37 // Boiling point
38 assertEquals(100.0, Computations.toCelsius(212.0), 0.001);
39 }
40 }